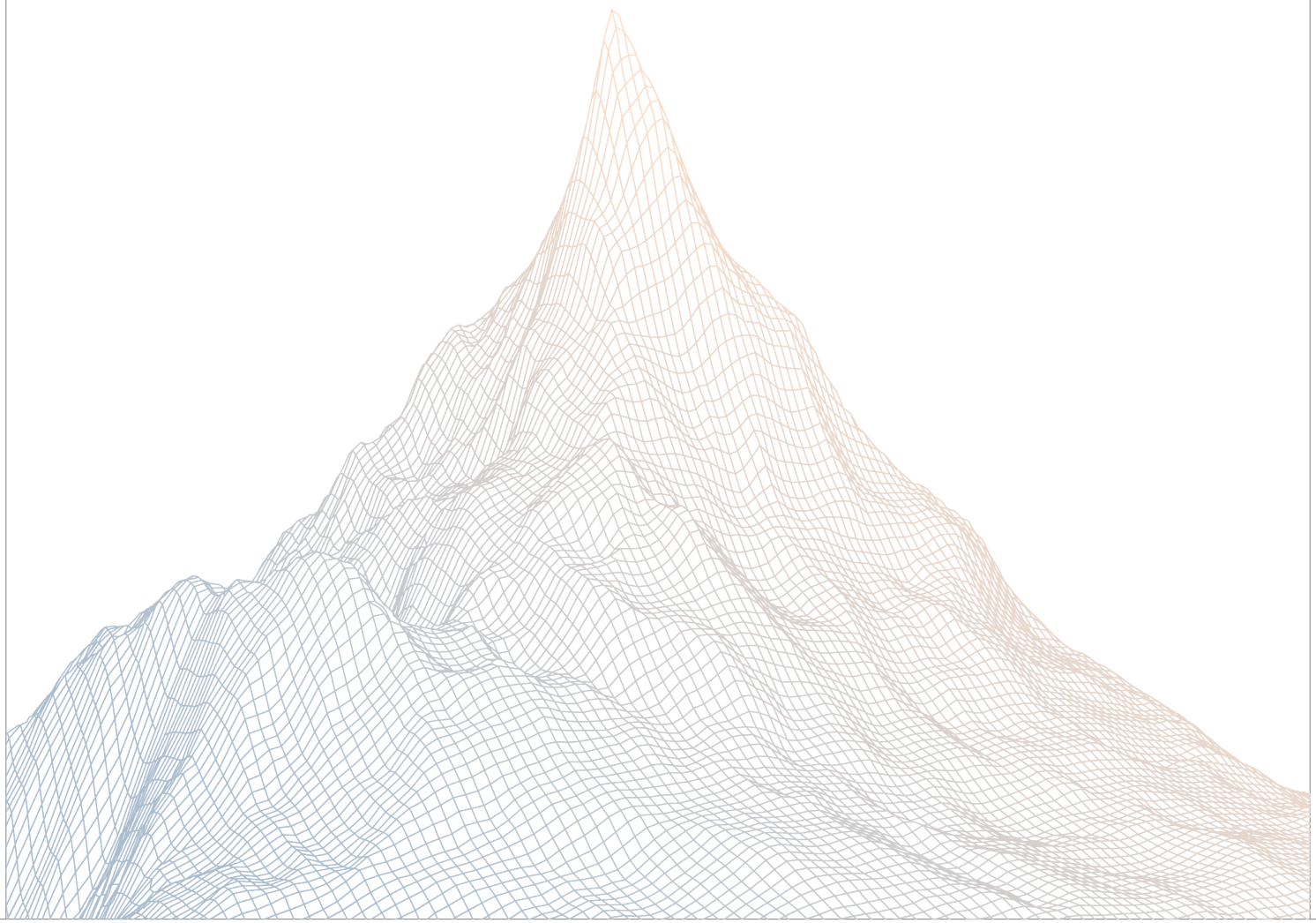
Zenith

Kite AI

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: March 24th to April 3rd, 2026

AUDITED BY: ast3ros
said

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About Kite AI 4

2.2 Scope 4

2.3 Audit Timeline 6

2.4 Issues Found 6

3 Findings Summary 6

4 Findings 7

4.1 Medium Risk 8

4.2 Low Risk 8

4.3 Informational 17

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Kite AI

Kite enables AI agents to autonomously transact at scale with cryptographic safety and native x402 compatibility—solving the infrastructure crisis imprisoning the agent economy today.

2.2 Scope

The engagement involved a review of the following targets:

Target	contracts-external
Repository	https://github.com/gokite-ai/contracts-external
Commit Hash	`3bdd766ecdf7ee80113ef0aa17e8c38ed18b18c2`

Files	`contracts/validator-manager/KiteStakingManager.sol`
	`contracts/validator-manager/StakingManager.sol`
	`Pause / Unpause functionality for Validator and Delegator registration`
	`Force remove validator functionality`
	`contracts/token/KiteNativeOFTAdapter.sol`
	`contracts/token/KiteOFTWithPausable.sol`
	`contracts/token/WKITE.sol`

Target	Mitigation Review
Repository	https://github.com/gokite-ai/contracts-external
Commit Hash	`c0f5a7010226de4dceb9722e51e0c6ee62662d0a`

Files	`contracts/validator-manager/KiteStakingManager.sol`
	`contracts/validator-manager/StakingManager.sol`
	`Pause / Unpause functionality for Validator and Delegator registration`
	`Force remove validator functionality`
	`contracts/token/KiteNativeOFTAdapter.sol`
	`contracts/token/KiteOFTWithPausable.sol`
	`contracts/token/WKITE.sol`

2.3 Audit Timeline

March 24, 2026	Audit start
April 3, 2026	Audit end
April 15, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	6
Informational	2
Total Issues	**8**

3

Findings Summary

ID	Description	Status
L-1	Admin can retroactively increase minimumStakeDuration to trap existing delegators	Resolved
L-2	forceRemoveValidator can revert due to churn rate limit	Acknowledged
L-3	_adminForceRemoveValidator allows a validator to consume churn budget twice in the same period	Acknowledged
L-4	Daily outbound limit can be griefed by token recycling	Acknowledged
L-5	Reward calculator replacement causes permanent staking reward loss	Acknowledged
L-6	Pausing blocks inbound bridge settlements in Kite	Acknowledged
I-1	Vault change orphans pending reward debts with no migration safeguard	Acknowledged
I-2	Inbound bridge to non-payable recipient permanently locks native KITE with no recovery mechanism	Acknowledged

4

Findings

4.1 Medium Risk

A total of 0 medium risk findings were identified.

4.2 Low Risk

A total of 6 low risk findings were identified.

[L-1] Admin can retroactively increase `minimumStakeDuration` to trap existing delegators

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingManager.sol#L1534-L1536

Description:

The delegator-initiated removal path checks `block.timestamp < delegator.startTime + $._minimumStakeDuration` using the mutable global `_minimumStakeDuration`, while the validator-initiated path uses per-validator immutable `minStakeDuration`. An admin can increase `_minimumStakeDuration` after delegators have already staked, retroactively extending their lock period. For example, a delegator who staked when `minimumStakeDuration = 14 days` expecting to unstake after 14 days could be trapped if admin increases it to `365 days`.

```
// Delegator  uses MUTABLE global
if (block.timestamp < delegator.startTime + $._minimumStakeDuration) {
    revert MinStakeDurationNotPassed(uint64(block.timestamp));
}

// Validator  uses IMMUTABLE per-validator value
if (block.timestamp < validator.startTime
    + $._posValidatorInfo[validationID].minStakeDuration) {
    revert MinStakeDurationNotPassed(uint64(block.timestamp));
```

```
}
```

Recommendations:

Consider also using `_posValidatorInfo[validationID].minStakeDuration` when checking the staking duration during delegator-initiated removal.

Kite AI: Resolved with `@d6c312490b ...`

Zenith: Verified.

[L-2] forceRemoveValidator can revert due to churn rate limit

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingManager.sol#L838-L892

Description:

`_adminForceRemoveValidator` calls `$._manager.initiateValidatorRemoval`, which internally calls `_checkAndUpdateChurnTracker`. If the current churn budget is exhausted, `_checkAndUpdateChurnTracker` reverts with `MaxChurnRateExceeded`, causing the entire admin force removal to revert.

```
    function _adminForceRemoveValidator(...) internal {
        StakingManagerStorage storage $ = _getStakingManagerStorage();
>>>     $._manager.initiateValidatorRemoval(validationID); // can revert if
    churn limit is hit
        ...
    }
```

This is a limitation for an emergency admin function when the churn budget is exhausted and blocks the call.

Recommendations:

Introduce a churn exempt path for admin initiated removals.

Kite AI: Acknowledged. In practice, if the churn budget is temporarily exhausted, the admin can execute the force removal after the current churn period resets. This introduces a bounded delay but does not prevent the removal from eventually completing.

[L-3] _adminForceRemoveValidator allows a validator to consume churn budget twice in the same period

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingManager.sol#L838-L892

Description:

The protocol enforces an invariant that `minimumStakeDuration ≥ churnPeriodSeconds`, which guarantees that a validator cannot complete removal and re-register within the same churn period, ensuring their stake weight counts against the churn budget at most once per period.

`_adminForceRemoveValidator` bypasses the `minStakeDuration` check entirely. Because stake is returned to the validator owner upon `completeValidatorRemoval`, the validator can immediately re-register with the same stake amount before the current churn period expires. This results in the same stake weight being counted against the churn budget twice in a single period, once as a removal and once as a new registration, violating the documented churn invariant.

If the validator's weight represents a large fraction of the total, this could push total churn closer to or past the `maximumChurnPercentage` limit, potentially blocking legitimate registrations.

Recommendations:

Document that `forceRemoveValidator` intentionally breaks the `minimumStakeDuration ≥ churnPeriodSeconds` invariant and note the churn budget implication.

Kite AI: Acknowledged. `adminForceRemoveValidator` is reserved for removing malicious validators, where the admin can choose to forfeit their rewards via the forfeitRewards parameter. A force-removed validator has no rational incentive to re-register. Even in the unlikely case of re-registration, the impact is limited to a tighter churn budget that temporarily blocks new registrations — no funds are at risk.

[L-4] Daily outbound limit can be griefed by token recycling

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- KiteNativeOFTAdapter.sol#L151-L175

Description:

The global daily outbound limit can be exhausted by a single malicious actor through token recycling, denying service to legitimate users at minimal cost.

Because `currentDayOutbound` tracks total volume across all senders, an attacker holding a big amount of KITE can repeatedly bridge tokens out and back, paying only LayerZero fees each round trip to consume the entire daily quota and block the send function for all non-exempt users until the next UTC day reset.

```
    function send(
        SendParam calldata _sendParam,
        MessagingFee calldata _fee,
        address _refundAddress
    ) public payable virtual override whenNotPaused returns
    (MessagingReceipt memory msgReceipt, OFTReceipt memory oftReceipt) {
        ...

            uint256 actualAmount = oftReceipt.amountSentLD;
            uint256 newOutbound = currentDayOutbound + actualAmount;
>>>         if (newOutbound > dailyOutboundLimit) {
                uint256 remaining = dailyOutboundLimit > currentDayOutbound
                    ? dailyOutboundLimit - currentDayOutbound
                    : 0;
                revert DailyOutboundLimitExceeded(actualAmount, remaining);
            }
            currentDayOutbound = newOutbound;
        }
    }
```

Recommendations:

- Cap net remote exposure instead of cumulative daily outbound volume OR
- Consider introducing a fee or max amount per send to raise the cost of griefing

Kite AI: Acknowledged. Each round trip costs LayerZero fees, and the attacker has no financial incentive, they gain nothing while losing money on every cycle. We believe the severity should be Low/Low given the cost-to-impact ratio.

[L-5] Reward calculator replacement causes permanent staking reward loss

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingManager.sol#L1987-L1995

Description:

When the owner calls `updateRewardCalculator()` at `KiteStakingManager.sol`, the new calculator is set immediately via `_updateRewardCalculator()` :

```
function _updateRewardCalculator(IRewardCalculator newRewardCalculator)
    internal {
    StakingManagerStorage storage $ = _getStakingManagerStorage();
    $._rewardCalculator = newRewardCalculator;
}
```

There is no intermediate step to claim accrued rewards under the old formula. The reward calculation in `_claimActiveValidatorRewards()`:

```
uint256 stakingReward = $._rewardCalculator.calculateIncrementalReward({
    stakeAmount: weightToValue(validator.startingWeight),
    lastClaimTime: lastClaimTime,    // @audit - set during LAST claim under
    old calculator
    currentTime: currentTime,         // @audit - now
    ...
});
```

The new calculator retroactively reprices the entire unclaimed period from `lastClaimTime` to `currentTime`. If the new calculator has a different rate, all unclaimed rewards change, potentially less (loss to stakers) or more (windfall at protocol's expense).

Recommendations:

Consider to force a global harvest (or snapshot pending rewards into `_redeemableValidatorRewards`) before swapping.

Kite AI: Acknowledged. We have a batch claim mechanism in place, so before any reward calculator update, we would trigger a global harvest first to settle all accrued rewards under the current formula.

[L-6] Pausing blocks inbound bridge settlements in Kite

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- KiteOFTWithPausable.sol#L43-L45

Description:

The Kite contract inherits `ERC20Pausable` and routes all token movements through it via `_update`:

```
function _update(address from, address to, uint256 value)
    internal override(ERC20, ERC20Pausable) {
    super._update(from, to, value);
}
```

Because `ERC20Pausable._update` reverts when paused, every token movement is blocked when the contract is paused, including inbound bridge settlements (`_credit`), which mint tokens to the recipient. It means pausing Kite creates a window where in-flight transfers from Kite mainnet are stuck: the tokens were already burned on the source chain but cannot be minted on this chain until the contract is unpaused, potentially stranding user fund utill the contract is unpaused.

Recommendations:

Mirror the pattern already used in KiteNativeOFTAdapter by overriding `_send` and applying `whenNotPaused` on the send side, while leaving `_credit` untouched.

Kite AI: Acknowledged. We've evaluated this and consider the risk acceptable, pausing is an admin-only emergency action and would only be triggered in extreme scenarios, and users' funds are not at risk since they can be credited once the contract is unpaused.

4.3 Informational

A total of 2 informational findings were identified.

[I-1] Vault change orphans pending reward debts with no migration safeguard

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- KiteStakingManager.sol#L368-L376

Description:

`updateRewardVault` swaps `$.rewardVault` to a new address but does nothing about accumulated reward debts stored in `_redeemableValidatorRewards` and `_redeemableDelegatorRewards`. These mappings live in `StakingManager` storage and record amounts owed. When users later call `claimValidatorRewards` or `claimDelegatorRewards`, the `_reward` function reads the current vault and attempts `vault.distributeReward`. If the new vault is unfunded, the balance check fails, `_reward` returns `false`, and the claim reverts with `RewardClaimFailed`.

The old vault retains the funds but the protocol no longer points to it. There is no on-chain enforcement that the new vault covers outstanding debts.

Recommendations:

Consider adding a check to ensure the funds have already been migrated to the new vault.

Kite AI: Acknowledged. updateRewardVault is an admin-only operation under multisig control. The operational procedure ensures funds are migrated to the new vault before the switch.

[I-2] Inbound bridge to non-payable recipient permanently locks native KITE with no recovery mechanism

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- KiteNativeOFTAdapter.sol

Description:

`NativeOFTAdapter._credit()` sends native KITE via a low-level call:

```
// NativeOFTAdapter.sol
(bool success, bytes memory data) = payable(_to).call{value: _amountLD}("");
if (!success) {
    revert CreditFailed(_to, _amountLD, data);
}
```

If `_to` is a contract that cannot accept native tokens (no `receive()/fallback()`, or one that reverts), the entire `_lzReceive()` call reverts. `KiteNativeOFTAdapter` inherits this behavior without overriding `_credit` to add a fallback mechanism.

Recommendations:

Consider add a fallback mechanism (e.g., wrapping to WKITE and sending the ERC20 instead, or storing credits in a mapping for pull-based claiming).

Kite AI: Acknowledged. In practice, cross-chain bridge recipients are typically EOAs or contracts capable of receiving native tokens. The responsibility lies with the sender to specify a valid recipient.